AI Changes the Delivery System and the Product Itself

PROBLEM → HYPOTHESIS → BUILD → VERIFY → REFRAME





HOW WE BUILD

How should we rebalance the SDLC?
As AI changes the work across delivery.

WHAT WE BUILD

Can model judgment remain useful and bounded?
Observable, correctable, worth the full cost.

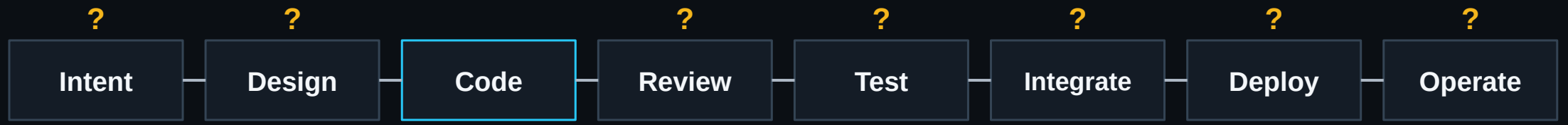
Vitalii Oborskyi Head of Delivery & PMO · Author of Uncertainty Architecture

DevCon Romania · 4 November 2026

vitaliioborskyi.ai

We Accelerated a Step. Not the System.

Faster generation is not the same as faster delivery.



Thought experiment: comparable changes/month; all admitted; fixed downstream capacity.

	Generated	Review capacity	Integration	BACKLOG
BEFORE Balanced flow	8	8	8	0 no accumulation
AFTER AI Other stages have not sped up	100	8	8	+92 waiting / month

WHAT IF NOT?

USER: adoption and feedback → intent

Limit admitted work: WIP stays bounded; excess demand waits outside the workflow.

The Comprehension Gap Is an Ownership Risk

A NARROW VIEW

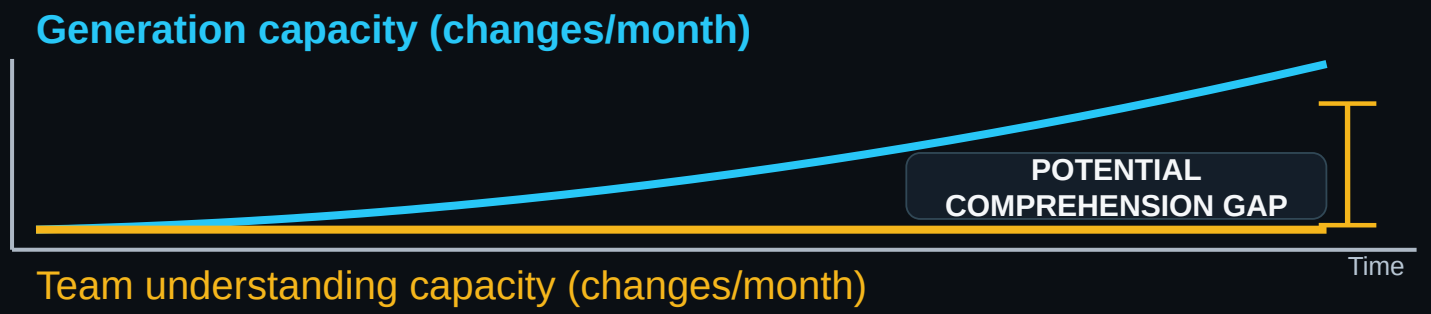
A quality gate that slows delivery

→

A SYSTEMIC VIEW · Code review also supports

- Knowledge transfer
- Shared understanding
- Ownership & accountability
- Operability & recovery
- Architecture & maintainability

Zoom in: CODE GENERATION ↔ REVIEW



WHAT IF NOT?

Can we generate faster?
YES

Can understanding keep pace?
NOT IN THIS SCENARIO

Can we explain, maintain and recover it?

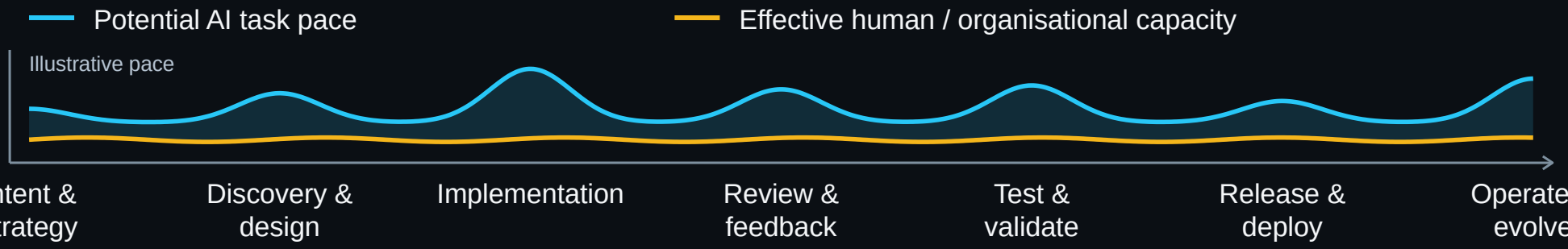
THE CONSEQUENCE · A gap in understanding can weaken competent system ownership
RESPONSIBILITY REMAINS · OWNERSHIP: ABILITY + AUTHORITY TO EXPLAIN · MAINTAIN · RECOVER.

AI EXPERIMENT · 69 PARTICIPANTS · TASK COMPLETION ↑ / CODE UNDERSTANDING ↓

Martin-Lopez et al. · IEEE TSE 2026 · 14 professional developers.

Short tasks; individual understanding, not team ownership. Graph illustrative.

Zoom out: the comprehension gap can affect every SDLC node



The gap creates pressure to rebalance

Lifecycle illustration: no measured values, stage rankings or throughput estimates.

What Are We Actually Measuring?

Activity, delivery, stability and maintainability can move differently.

ACTIVITY · RELEASES · MARKETPLACE SIGNALS

NBER · DIFFERENT SAMPLES / TOOL STAGES · ESTIMATES VS BASELINE

25.5×	3.4×	1.3×
CHANGED LOC · ALL	COMMITS · ALL	RELEASES · NO ASYNC

MARKETPLACES · NEW SUPPLY VS EARLY USE

~2×	≈ FLAT	78 → 87%
NEW iOS APPS	EARLY RATINGS	APPS <10 RATINGS

Android downloads rose modestly; usage proxies differ.

FLOW · CHEN–STRATTON 2026 · 718 FIRMS

+23%	+49%
PRs / ACTIVE WORKER-MONTH	PR → MERGE (ELAPSED)

Calendar elapsed time, not reviewer hours; completion gains uncertain.

SUPPORTING · DORA 2025 · SURVEY n=4,867

AI ADOPTION ↔ THROUGHPUT + INSTABILITY

Self-reported effects; survey association.

Survey association; instability = failed changes + unplanned fixes.

SUPPORTING · AGARWAL 2026 · 1,197 REPOS

COMPLEXITY	WARNINGS
CHANGES OBSERVED	AGENT-FIRST COHORT

Matched OSS cohorts; AF = agent first. Proxies ≠ defects or cost.

MAINTAINABILITY · GITCLEAR 2026 · 623M CHANGES

MOVED CODE	−70%	2023 → YTD26
DUPL. BLOCKS	+81%	2023 → YTD26

Duplication per 1M changed lines; no causal claim.

No one productivity number. Measure useful outcomes, stability, maintainability and full cost.

Different methods and samples; no common funnel. Async release effect not separately estimated.

Rebalancing a Node Affects More Than Flow

Nodes exchange artifacts and knowledge. Six hypotheses to test.

SPEED & FULL COST	DELIVERY & PRODUCT QUALITY	KNOWLEDGE & OWNERSHIP
Small batches + WIP limits Limit admitted work and queue growth. Where does waiting move?	**1 · Agentic review + test oracles** Scale checks; ground evidence. Shared blind spots? Who learns?	**2 · Living architecture + docs** Explain decisions and code changes. Who verifies it? Who learns?
Spec-driven development Durable intent, specs and contracts. Compare with a baseline. Do specs stay valid and useful?	**3 · Disposable implementation** Regenerate isolated, bounded features. State and integration limits?	**Repository intelligence** Map dependencies, changes and owners. Is the map grounded and current?

ONE UNIVERSAL AI-NATIVE SDLC?
Maybe context-specific operating models.

CONTEXT: domain · size · complexity · legacy · criticality · team · cost
AGENT SCOPE: vary by SDLC node — including none

RESPONSIBILITY REMAINS. CAN THE TEAM STILL UNDERSTAND AND RECOVER?

REVIEW: quality + knowledge for ownership · TRACK: delivery, stability, recovery and cost

ACT II / WHAT WE BUILD

When Model Judgment Enters Runtime

Account recovery: acceptable advice, prohibited disclosures, authorized actions

EXPLICIT RULES / SIMPLIFIED PATH

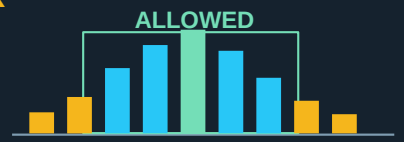
$$y = f(x)$$

Same relevant inputs + state + version

MODEL JUDGMENT / CONDITIONAL BEHAVIOR

$$y \sim P(y \mid x, c)$$

c = context + model configuration



ENGINEERING	EXPLICITLY ENCODED BEHAVIOR	MODEL-DEPENDENT BEHAVIOR SPACE
REQUIREMENT	Specified behavior, constraints and outcomes	Allowed outcomes + prohibited disclosures
BUG	Expected path fails / wrong output	Wrong advice or disclosure; severity matters
DoR	Requirements clear enough to build	Scope, risks and evidence plan ready
DoD	Increment meets agreed quality	Meets criteria; operating approval separate

DEVOPS / MODEL UNCERTAINTY

Model decisions and context can change during operation

NEW CLASS / THINKING SYSTEMS

More cognitive work can become viable to delegate.

Rollout is not proof. Observe outcomes AND controls; narrow, stop or rethink.

ACT II / RELEASE ≠ END OF VALIDATION

The Rollout Ends. The Hypothesis Doesn't.

A deployed model still has to earn and keep its authority under changing conditions.

EVALUATION / ILLUSTRATIVE n = 200

196	3	1
ACCEPTABLE	WRONG STEPS	PRIVACY LEAK

98% acceptable ≠ approval

A prohibited disclosure blocks approval for this scope

PROGRESSIVE EXPOSURE / IF RISK ALLOWS

INDEPENDENT EVALUATION › SHADOW NO ACTION › CANARY BOUNDED › OPERATE WITH LIMITS

A canary reduces exposure. It is not a safety certificate.

Preventive controls are required before live exposure.

SAME APP SHA ≠ SAME OPERATIONAL CONDITIONS

Model / provider route · retrieved knowledge · policies · users · responder capacity

AFTER ROLLOUT: MONITOR THE WHOLE CONTROL SYSTEM

SERVICE
Latency · errors · uptime
Resource use · full cost

BEHAVIOR
Useful outcomes
Wrong / forbidden advice

CONTROLS
Gate failures · bypasses
Escalation · fallback

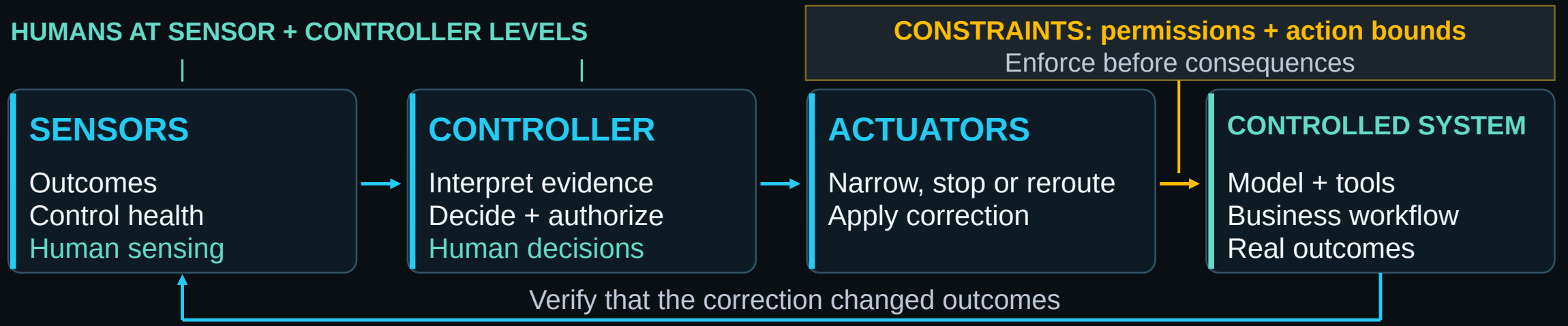
PEOPLE / CAPACITY
Availability · workload
Queue · response time

CONTINUE · NARROW AUTHORITY · MANUAL FALLBACK · STOP + REASSESS

Rollback cannot undo a disclosure. A passed canary does not retire the hypothesis.

Engineer the Whole Control System.

Can this control loop remain useful, bounded, correctable and worth the full cost?



When people fill these roles, operators become an architectural dependency

Availability, competence, workload, response time and authority

A successful demo is not enough

Does this control loop fit the business case and context?

Detect failures? Prevent or limit harm? Intervene in time? Full cost acceptable?

Evidence insufficient? Fund discovery, narrow scope, redesign or no-go.

AI Changes Both Sides. The Engineering Must Follow.

Two research directions. One question: what if not?

HOW WE BUILD

Rebalance the SDLC.

FLOW · QUALITY · KNOWLEDGE

Measure outcomes,
recovery and cost.

THE SUBPRIME CODE CRISIS

Systems thinking · Theory of Constraints

WHAT WE BUILD

Engineer Thinking Systems.

BOUNDARIES · EVIDENCE · ACTION

Observe outcomes.
Retain authority to act.

UNCERTAINTY ARCHITECTURE

Systems thinking · Control Theory

BOTH PROJECTS · PUBLIC RESEARCH

https://vitaliioborskyi.ai/

Research · Writing · Contact

QUESTIONS?

DEVOPS / SRE	ENGINEERS / ARCHITECTS	CTO / DELIVERY / PRODUCT
Observe · intervene · recover	**Define bounds · test controls**	**Assign owners · fund evidence**